

Cairo University
Faculty of Engineering
Computer Engineering Department

Data Structures and Algorithms

Sheet #2

Linked List

[Assumptions]:

This sheet presents some general operations that can be made on a general linked list (NOT specific for Bag ADT or List ADT).

All sheet problems assume that a linked list consists of nodes where each node has "data" (item) and pointer "next".

The linked list itself is represented by a pointer to list head.

Part1: Tracing Problems

1.

- What is the output of the following algorithm if L =[30, 5, 20, 9, 2, 17]
- What does this algorithm do?
- What would happen if L is passed as an empty list? If there is a problem, suggest a fix.

Input: Linked List L (L is a pointer to the first node)

Algorithm Algo(L)

```
ptr = L
V = ptr->data
ptr = ptr->next
while (ptr)
    if (ptr->data < V)
        V = ptr->data
    ptr = ptr->next
Print V
```

2.

Input: Linked List L

Algorithm Algo(L)

```
if (!L) return
ptr = L
while (L && L->data < 0)
    ptr = L
    L = L->next
    delete ptr
prev= L
ptr = prev->next
while(ptr)
    if(ptr->data < 0)
        prev->next = ptr-> next
        delete ptr
    else
        prev=ptr
    ptr = prev->next
```

- Show the contents of the list L after the end of the algorithm in the following cases.
 - L =[30, 5, 20, 9, 2, 17]
 - L =[55, -60, 44, -9, -21, 15, -7]
 - L =[-55, -60, -9]
- Would any of the above cases result in a runtime error? If so, suggest a fix.
- What does the algorithm do?

3.

Input: Linked List L which is sorted ascendingly.

```

Algorithm Algo(L)
  IF(!L) return
  ptr = L
  while( ptr-> next )
    start = ptr->data
    end = ptr -> next -> data
    For X = start+1 to end-1
      R = new Node
      R->data = X
      R->next = ptr->next
      ptr->next = R
      ptr = ptr->next
  ptr = ptr -> next

```

- Show the contents of the list L after the end of the algorithm in the following cases.
 - L =[3, 5, 9,10]
 - L =[3,4]
 - L =[3]
- Would any of the above cases result in a runtime error? If so, suggest a fix.
- What does the algorithm do?

4.

Input: Linked Lists L1, L2

```

Algorithm Algo(L1, L2)
  ptr1 = L1   ptr2 = L2   L2 = NULL
  if (!ptr1)
    L1 = ptr2
    return
  while (ptr1->next)
    ptr1 = ptr1->next
  ptr1->next = ptr2

```

- Show the contents of L1 and L2 after the end of the algorithm in the following cases.
 - L1 =[30, 5, 90] and L2 =[4, 5, 12, 70]
 - L1 =[] //Empty and L2 =[4, 5, 12, 70]
 - L1 =[30, 5, 90] and L2 =[] //Empty
- What does the algorithm do?

5.

Input: Linked List L**Result:** algorithm result will be stored in L1 and L2

```

Algorithm Algo(L, L1, L2)
  slow = L      fast = L
  while ( fast != NULL AND fast->next != NULL)
    slow = slow->next
    fast = fast->next->next
  p = slow
  L1 = L
  L2 = p->next
  p->next = NULL

```

- Show the contents of L1 and L2 after the end of the algorithm in the following cases.
 - L =[30, 5, 90, 10, 17]
 - L =[30, 5, 90, 10]
- Is there a corner case that is not handled by the above algorithm? If so, suggest a fix.
- What does the algorithm do?

6.**Input:** Linked List L**Result:** algorithm result will be stored in L1 and L2**Algorithm** Algo(L, L1, L2)

```
L1 = L
L2 = L->next
p1 = L1
p2 = L2
while(p2)
    p1->next = p2->next
    p1 = p2
    p2 = p2->next
L = NULL
```

- Show the contents of L1 and L2 after the end of the algorithm if
L=[30, 5, 90, 10, 17, 55, 70]
- Is there a corner case that is not handled by the above algorithm? If so, suggest a fix.
- What does the algorithm do?

7.**Input:** Linked Lists L1 and L2**Result:** algorithm result will be stored in L**Algorithm** Algo(L1, L2, L)

```
p1 = L1    p2 = L2    L = L1
while(p1 && p2)
    t1 = p1->next
    t2 = p2->next
    p1->next = p2
    p2->next = t1
    p1 = t1
    p2 = t2
L1 = NULL
L2 = NULL
Return L
```

- To trace the above algorithm, suggest arbitrary data in both L1 and L2 then trace the algorithm and show the contents of L
- What would happen if one of the above lists is much longer than the other? How to fix.

Part2: Code Problems

- **Problems in this part are for self-training to practice coding on Linked Lists**
- **Not to be solved during the problem set session**
- **You may try them during lab sessions**
- **Answers will be provided to you for guidance**

1. Write an algorithm that traverses a list and **counts** the number of negative elements. Give an array and a linked list implementation.
2. Write an algorithm that finds the minimum value in a list. Give an array and a linked list implementation.
3. You are given two lists L and P containing integers sorted in ascending order. Write an algorithm **PrintLots(L,P)** that will print elements in L that are in positions specified by P. e.g. if P = 1, 3, 4, 6, the elements in positions 1, 3, 4, 6 in list L are printed. Give an array and a linked list implementation.
4. You are given a sorted list L. Write an algorithm that **completes** L to become a list of consecutive numbers by **inserting** the missing numbers between each two numbers in the original list. For example if L={2, 5, 7} in the above example, L will be{2, 3, 4, 5, 6, 7}. Give an array and a linked list implementation.
5. Write an algorithm that traverses a list implemented using a linked list and **deletes all** nodes whose keys are negative.
6. Write an algorithm that **appends** two lists L1 and L2 together. After executing the algorithm, the appended list should be in L1 while L2 should be empty. Use linked list implementation.
Note: you don't need to allocate/deallocate any new nodes.
7. Write an algorithm that **appends** a list to itself so that the resulting list contains two copies of the original list. For example if input list is L={10, 2, 14}, the result is L= {10, 2, 14, 10, 2, 14}. Use linked list implementation.
8. Write an algorithm that **swaps** (exchanges) two nodes in a list. The nodes are identified by numbers indicating their index. For example, to exchange nodes of index: 5 and 8, you would call *swap* (5, 8), the smallest index is passed first. If the exchange is successful, the algorithm returns true. If it encounters an error, such as an invalid node number, it returns false.
Use linked list implementation.
Note: this swapping can be made using temp data variable and can be used using just pointers manipulation without creating temp data variable. Try both methods and tell which is better if the stored data has a large size.
9. Write a function that **merges** two ordered lists L1 and L2 implemented as linked lists into one linked list. When the two lists are merged, the data in merged list are also ordered. Do not remove duplicates if found. Write a function for each case of the following:
 - a. The merged list should be a new list L3. L1 and L2 should be left unchanged.
 - b. Put the merged list in L1 and make L2 empty.[Constraint]: don't create or delete any new nodes.
10. Implement a function **AlternatingSplit(...)** that given a list L, splits it into two lists (L1 and L2) where the first node goes to L1, the second to L2, the third to L1, the fourth to L2 and so on. At the end of this function, the input list should be pointing to NULL.
[Constraint]: don't create or delete any new nodes.

11. Implement a function **AlternatingMerge(...)** that reverses what you made in the previous problem; merges generated L1 and L2 into L.
[Constraint]: do not create or delete any new nodes.
12. The **Josephus problem** is the following game:
N people numbered 1 to N are sitting in a circle. Starting at person 1, a hot potato is passed. After M passes, the person holding the hot potato is eliminated, the **circle** closes ranks, and the game continues with the person who was sitting after the one eliminated picking up the hot potato. The last remaining person wins. Write an algorithm to solve the Josephus problem for general M and N.
For example:
If N = 6 (players are p1 to p6) and M = 3,
the order of deletion is: {p4 -> p2 -> p1 -> p3 -> p6}, winner is p5.
13. **Implement** all the above problems in C++ and write an appropriate main function that calls your functions and tests them.
14. Write a program to create and display a circular linked list.
15. **Cumulative sum** Given a null-terminated linked list, in, create a new null-terminated linked list out, of the same length, such that node *i* of out contains the sum of the data in in's nodes up to and including node *i* of list in.